TableSplit

№ id	28
チェック	☐
機能カテゴリー	テキスト処理 ファイル処理
機能概要	CSV列の配列値を展開

TableSplit 取扱説明書

１．この機能でできること

CSVデータの中にある配列（リスト）を1つずつ展開して、分析しやすい形に変換できます。

できること：

- CSV内の配列データを行方向に展開（1つの配列を複数の行に分ける）
- CSV内の配列データを列方向に展開（1つの配列を複数の列に分ける）
- JSON形式で保存されている配列データも自動で認識して展開
- 配列が空の場合や配列でない場合も安全に処理

こんな場面で使えます：

- アンケート結果で「複数選択可」の項目を1行ずつ分けて集計したい
- 顧客データに「購入商品リスト」があり、商品ごとに分析したい
- APIから取得したデータに配列が含まれていて、Excelで扱いやすくしたい
- タグやカテゴリーが複数ある記事データを、タグ別に集計したい

必要なもの：

- `input_csv` `型：文字列` （例：CSV形式のテキストデータ）

出てくるもの：

- `output_csv` `型：文字列` （例：展開後のCSV形式のテキストデータ）

設定する項目：

- `column_name`: 展開したい列の名前（例："tags", "購入商品", "選択肢"）
- `mode`: 展開の方向を指定
  - `"row_split"`: 行方向に展開（1つの配列を複数行に分ける）
  - `"column_split"`: 列方向に展開（1つの配列を複数列に分ける）

２．基本の使い方

使用例1: アンケートの複数選択項目を行ごとに展開

やりたいこと： 「好きな果物」の複数選択アンケート結果を、果物ごとに1行ずつ分けて集計したい

```
# 元のCSV:
# name,favorite_fruits
# 田中,"[\"りんご\", \"バナナ\"]"
# 佐藤,"[\"みかん\"]"

Input(
    input_csv="name,favorite_fruits\n田中,[\"りんご\",\"バナ
ナ\"]\n佐藤,[\"みかん\"]"  # 型: 文字列
)

# 設定
Params(
    column_name="favorite_fruits",  # 型: 文字列 | 展開したい
列の名前
    mode="row_split"                # 型: 文字列 | 行方向に展
開
)

# 結果:
# name,favorite_fruits
# 田中,りんご
# 田中,バナナ
# 佐藤,みかん
Output(
    output_csv="name,favorite_fruits\n田中,りんご\n田中,バナナ
```

```
\n佐藤,みかん"  # 型: 文字列
)
```

使用例2: 購入商品リストを列ごとに展開

やりたいこと： 顧客の購入商品を「商品1」「商品2」のように列で分けて表示したい

```
# 元のCSV:
# customer_id,purchased_items
# C001,"[\"ノートPC\", \"マウス\", \"キーボード\"]"
# C002,"[\"タブレット\"]"

Input(
    input_csv="customer_id,purchased_items\nC001,[\"ノートPC\",\"マウス\",\"キーボード\"]\nC002,[\"タブレット\"]"  # 型: 文字列
)

# 設定
Params(
    column_name="purchased_items",  # 型: 文字列 | 展開したい列の名前
    mode="column_split"             # 型: 文字列 | 列方向に展開
)

# 結果:
# customer_id,purchased_items_0,purchased_items_1,purchased_items_2
# C001,ノートPC,マウス,キーボード
# C002,タブレット,,
Output(
    output_csv="customer_id,purchased_items_0,purchased_items_1,purchased_items_2\nC001,ノートPC,マウス,キーボード\nC002,タブレット,,"  # 型: 文字列
)
```

使用例3: ブログ記事のタグを行展開して分析

やりたいこと： ブログ記事に付いているタグを1つずつ分けて、タグごとの記事数を集計したい

```
# 元のCSV:
# article_id,title,tags
# A001,Python入門,"[\"プログラミング\", \"Python\", \"初心者向け\"]"
# A002,データ分析の基礎,"[\"Python\", \"データ分析\"]"

Input(
    input_csv="article_id,title,tags\nA001,Python入門,[\"プログラミング\",\"Python\",\"初心者向け\"]\nA002,データ分析の基礎,[\"Python\",\"データ分析\"]"  # 型: 文字列
)

# 設定
Params(
    column_name="tags",    # 型: 文字列 | 展開したい列の名前
    mode="row_split"       # 型: 文字列 | 行方向に展開
)

# 結果:
# article_id,title,tags
# A001,Python入門,プログラミング
# A001,Python入門,Python
# A001,Python入門,初心者向け
# A002,データ分析の基礎,Python
# A002,データ分析の基礎,データ分析
Output(
    output_csv="article_id,title,tags\nA001,Python入門,プログラミング\nA001,Python入門,Python\nA001,Python入門,初心者向け\nA002,データ分析の基礎,Python\nA002,データ分析の基礎,データ分析"
# 型: 文字列
)
```

使用例4: 複雑なデータ（辞書の配列）も展開可能

やりたいこと： 配列の中に辞書形式のデータがある場合も、そのまま文字列として展開したい

```
# 元のCSV:
# order_id,items
# O001,"[{\"name\":\"商品A\",\"price\":1000}, {\"name\":\"商品B\",\"price\":2000}]"

Input(
    input_csv="order_id,items\nO001,[{\"name\":\"商品A\",\"price\":1000},{\"name\":\"商品B\",\"price\":2000}]"  # 型: 文字列
)

# 設定
Params(
    column_name="items",   # 型: 文字列 | 展開したい列の名前
    mode="row_split"       # 型: 文字列 | 行方向に展開
)

# 結果（辞書はJSON形式で文字列化される）:
# order_id,items
# O001,"{"name":"商品A","price":1000}"
# O001,"{"name":"商品B","price":2000}"
Output(
    output_csv="order_id,items\nO001,{\"name\":\"商品A\",\"price\":1000}\nO001,{\"name\":\"商品B\",\"price\":2000}"  # 型: 文字列
)
```

３．他の機能との組み合わせ活用と注意点

よく使う組み合わせ

1. API取得データの前処理

流れ： API呼び出し → JSON→CSV変換 → TableSplit → データ分析

- **やること：** APIで取得した複雑なデータを分析用に整形する

- **使う機能：** APIレスポンスをCSV化した後、TableSplitで配列を展開すると、Excelやデータベースで扱いやすくなります

2. アンケート集計パイプライン

流れ： CSVアップロード → TableSplit → フィルタリング → 集計

- **やること：** 複数選択可のアンケート結果を選択肢ごとに集計する
- **使う機能：** TableSplitで行展開した後、フィルタ機能やグループ化機能を使うと、選択肢ごとの集計が簡単になります

3. タグ分析ワークフロー

流れ： データ取得 → TableSplit → 重複削除 → カウント集計

- **やること：** ブログ記事やSNS投稿のタグを分析して人気タグを把握する
- **使う機能：** TableSplitでタグを1行ずつ展開し、重複削除とカウント機能で頻出タグを特定できます

4. 商品購入分析

流れ： 購入履歴取得 → TableSplit → 商品マスタと結合 → 売上集計

- **やること：** 顧客ごとの購入商品リストを商品別の売上データに変換する
- **使う機能：** TableSplitで購入商品を展開後、商品マスタデータと結合することで詳細な分析が可能になります

気をつけること

⚠️ データ形式について

- **配列の記述方法**：配列データはJSON形式（`["item1", "item2"]`）で記述する必要があります
- **CSV内のJSON**：CSVファイル内にJSON形式の配列が文字列として保存されていても自動で認識されます
- **おすすめ：** 配列データがカンマ区切りのテキストの場合は、先に配列形式（JSON）に変換してから使用すると正確です

💡 行展開と列展開の使い分け

- **行展開（row_split）を使うとき：**
  - 配列の要素数が行ごとに異なる場合

    - 後で集計やフィルタリングをする予定がある場合
    - データ分析やピボットテーブルで使う場合
- **列展開（column_split）を使うとき：**
    - 配列の要素数が固定（または最大値が決まっている）場合
    - Excelのような表形式で見やすくしたい場合
    - レポート出力や表示用のデータを作る場合
- **例：** アンケート結果の集計には「row_split」、レポート作成には「column_split」が適しています

📊 データ量に関する注意

- 行展開（row_split）を使うと、元のデータ行数 × 配列の要素数分だけ行が増えます
- 例：100行のデータで、各行に平均5個の配列要素がある場合、結果は約500行になります
- 大量のデータを処理する場合は、事前にサンプルデータで動作確認することをおすすめします

⏱️ 処理時間について

- 小規模データ（数百行）：数秒以内
- 中規模データ（数千行）：数秒〜数十秒
- 大規模データ（数万行以上）：1分以上かかる場合があります
- 配列要素が多いほど処理時間が長くなります

✅ エラーが出たら

- **「指定した列が見つかりません」** → `column_name` のスペルミスや大文字小文字の違いを確認してください
- **「結果が空になる」** → 元のCSVデータが空か、配列形式が正しくない可能性があります。データ形式を確認してください
- **「配列が展開されない」** → 配列データがJSON形式（`["item1", "item2"]`）になっているか確認してください。カンマ区切りのテキストの場合は、先に配列形式に変換が必要です

🔄 列展開時の列名

- 列展開（column_split）を使うと、元の列名に `_0`, `_1`, `_2` のように連番が付きます
- 例：`tags` という列を展開すると、`tags_0`, `tags_1`, `tags_2` になります
- 配列要素数が行によって異なる場合、短い行の列は空白になります

💾 空の配列や配列でないデータの扱い

- 空の配列 `[]` の場合：行展開では1行作成され、その列は空白になります
- 配列でないデータの場合：そのまま変更せずに出力されます
- JSONパースエラーが発生した場合：元のデータがそのまま保持されます（処理は中断しません）